Федеральное государственное автономное образовательное учреждение высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Отчёт по задачам раздела №3

Структуры данных

Выполнил: Караганов Павел

Группа: P3210

Проверил: Косяков Михаил Сергеевич

г. Санкт-Петербург,

2026

I. Машинки

Задача решается как оптимальное управление ограниченным «полом» из **k** машинок. Если нужно освободить место, удаляется машинка, чьё следующее использование произойдёт позже всех (или не произойдёт вовсе).

Идея реализации:
- для каждой позиции заранее хранится индекс следующего появления такой же машинки;
- в **set** поддерживается текущее множество машинок на полу, отсортированное по времени следующего использования;
- при отсутствии машинки на полу и полном наборе удаляется элемент с максимальным **next**, затем добавляется текущая машинка.

Асимптотика:
- время: $O(p \log k)$, где p — длина последовательности;
- память: $O(n + p)$.

Почему так:
- каждое действие с **set** (**insert**, **erase**, получение крайнего элемента) стоит $O(\log k)$;
- на каждом из p шагов выполняется константное число таких операций;
- значит общее время $O(p \log k)$;
- дополнительно хранятся массивы о вхождениях машинок и сама последовательность, что даёт $O(n + p)$ памяти.

Почему это оптимально:
- если в момент вытеснения убрать не «самую позднюю» машинку, а любую другую, можно ухудшить ответ на следующем шаге;
- выбор машинки с максимально поздним следующим использованием является жадным шагом, который локально не ухудшает и глобально минимизирует число загрузок;

```
#include <algorithm>
#include <deque>
#include <iostream>
#include <map>
#include <queue>
#include <set>
#include <stack>
#include <string>
#include <unordered_map>
#include <vector>

#define INF 1000000000
#define ll long long

using namespace std;

struct Car {
  int next;
  int val;
  int past;

  bool operator<(const Car& other) const {
    return next < other.next;
  }
```

```
};

int main() {
  int n, p, k;
  cin >> n >> k >> p;

  vector<int> parents(n + 1, -1);
  vector<Car> seq(p, {INF, 0, -1});

  for (int i = 0; i < p; i++) {
    cin >> seq[i].val;

    if (parents[seq[i].val] == -1) {
      parents[seq[i].val] = i;
    } else {
      seq[parents[seq[i].val]].next = i;
      seq[i].past = parents[seq[i].val];
      parents[seq[i].val] = i;
    }
  }

  set<Car> floor;
  vector<bool> playing(n + 1, false);
  int ans = 0;

  for (int i = 0; i < p; i++) {
    Car cur = seq[i];

    if (playing[cur.val]) {        // машинка уже на полу и Петя ей уже играет
      floor.erase(seq[cur.past]);  // тут и на слеудующей строке просто обновляем время
появления
      floor.insert(cur);
    } else {  // машинки нет, надо узнать что удалить

      // удаляем самую дальнюю машинку (машинку с максимальным next)
      if ((int)floor.size() == k) {
        Car delet = floor.extract(--(floor.end())).value();
        floor.erase(delet);
        playing[delet.val] = false;
      }

      // добовляем текущую машинку
      floor.insert(cur);
      playing[cur.val] = true;

      ans++;
    }
  }

  cout << ans;
}
```

J. Гоблины и очереди

Нужно поддерживать очередь с операциями добавления в конец (операция с +), добавления в середину (операция с *) и удаления из начала (операция с -). Решение использует две очереди: левая часть хранит начало очереди, правая — оставшуюся часть.

Идея реализации:
- после каждой операции поддерживается баланс размеров: $|\mathrm{d2}| \leq |\mathrm{d1}| + 1$;
- добавление в конец идёт в **d2**, добавление в середину — в начало **d2**;
- удаление выполняется из начала **d1**, затем снова выполняется балансировка.

Асимптотика:
- время: $O(t)$, где t — число операций;
- память: $O(t)$ в худшем случае.

Почему так:
- каждая операция над **deque** (**push_front**, **push_back**, **pop_front**) амортизированно $O(1)$;
- балансировка переносит не более одного элемента за операцию, тоже $O(1)$;
- поэтому суммарно за t запросов получаем $O(t)$;
- в худшем случае все добавленные элементы одновременно находятся в двух деках, что даёт линейную память.

Почему это оптимальное решение:
- по условию требуется онлайн-обработка команд, и каждая команда должна отрабатывать быстро без пересборки структуры;
- разбиение очереди на две деки с инвариантом баланса обеспечивает $O(1)$ амортизированно на каждую операцию, что является наилучшим возможным порядком для этой модели;
- решение использует только необходимые структуры и сохраняет линейную память относительно числа элементов.

```
#include <iostream>
#include <map>
#include <stack>
#include <string>
#include <vector>
#define INF INT_MAX
#define ll long long

using namespace std;

deque<int> d1, d2;

void central() {
  if (d1.size() + 1 == d2.size()) {
    d1.push_back(d2.front());
    d2.pop_front();
  }
}

void push_back(int num) {
  d2.push_back(num);
  central();
}

void push_mid(int num) {
```

```
  d2.push_front(num);
  central();
}

void pop() {
  cout << d1.front() << '\n';
  d1.pop_front();
  central();
}

int main() {
  int t;
  string s;

  cin >> t;

  for (; t > 0; t--) {
    cin >> s;

    if (s[0] == '+') {
      cin >> s;
      push_back(stoi(s));
    } else if (s[0] == '*') {
      cin >> s;
      push_mid(stoi(s));
    } else {
      pop();
    }
  }
}
```

K. Менеджер памяти-1

Реализация основана на **неявном дереве отрезков** по диапазону адресов [1; n], где n может быть очень большим (до $2^{\{31\}} - 1$). В вершине хранятся три значения:

- **pref** — длина максимального свободного префикса сегмента;
- **suff** — длина максимального свободного суффикса;
- **max** — длина максимального свободного подотрезка внутри сегмента.

Идея решения:

- хранить состояние памяти в неявном дереве отрезков, где каждая вершина отвечает за свой диапазон адресов;
- по полям **pref**, **suff** и **max** быстро понимать, можно ли выделить непрерывный блок нужной длины;
- для запроса выделения брать самый левый подходящий блок, для освобождения — перекрашивать ранее выделенный диапазон в свободный;
- чтобы корректно освобождать по номеру запроса, сохранять границы успешных выделений в массиве **history**.

Также используется ленивый флаг **busy** (для проталкивания):

- **1** — весь сегмент свободен;
- **0** — весь сегмент занят;
- **−1** — нет отложенного присваивания.

Операции:

- запрос **x > 0** (выделить **x** подряд) — спускаемся по дереву и находим самый левый отрезок длины не меньше **x** (**get_start**), затем красим его в «занято» (**update**);
- запрос **x < 0** (освободить результат **-x**-го выделения) — берём сохранённые границы из **history** и красим их в «свободно».

Почему решение проходит по памяти (64 МБ):

- дерево строится **динамически**, вершины создаются только при реальном заходе в подотрезок;
- глубина дерева не больше $\lceil \log_2 n \rceil \leq 31$, поэтому на одну операцию создаётся ограниченное число новых вершин;
- размер одной вершины в этой реализации — порядка 24 байт (поля: 3 ui + 2 int + 1 signed char, с учётом выравнивания = 3 * 4 + 2 * 4 + 1 * 1 + 3 = 24);
- при **MAX_NODES = 2 100 000** массив **tree** занимает примерно $2.1 * 10^6 * 24 \approx 50.4$ МБ;
- массив **history** при $m \leq 10^5$ добавляет около 1–2 МБ;
- вместе с прочими переменными остаётся безопасный запас до лимита 64 МБ.

Почему выбрана именно константа **MAX_NODES = 2 100 000**:

- константа выбрана **от лимита памяти**, а не от теоретического максимума числа вершин;
- если взять заметно меньше, есть риск упереться в лимит узлов;
- если взять заметно больше, можно превысить лимит 64 МБ только на одном массиве **tree**;
- значение $2.1 * 10^6$ даёт около 50.4 МБ под вершины и оставляет 13 МБ запаса под **history**, стек и прочий runtime-оверхед;

Почему решение проходит по времени (1 секунда):

- каждая операция **update/get_start** работает за $O(\log n)$;
- при $n \leq 2^{\{31\}} - 1$ имеем $\log_2 n \leq 31$, то есть фактически очень короткий путь в дереве;
- на один запрос выполняется константное число таких проходов, поэтому суммарно для m запросов: $O(m \log n)$;
- при $m \leq 10^5$ это порядка нескольких миллионов простых операций над массивом, что для C++ укладывается в 1 секунду.

Асимптотика:

- время: $O(m \log n)$;
- память: $O(S + m)$, где S — число реально созданных вершин неявного дерева (в худшем случае ограничено **MAX_NODES**), а при данных лимитах — не более выделенного статического массива.

NB: Изначально я писал дерево отрезков (ДО) на структурах и указателях (как решение следующей заадчи), не задумываясь о памяти, но такое решение выдало 4 RE из-за переполнения, поэтому пришлось переписать ДО на массивы с выделением сразу максимального числа узлов. Благодаря этому я понял почему написание ДО на массиве счиается лучше чем на структурах.

```
#include <algorithm>
#include <deque>
#include <iostream>
#include <map>
#include <queue>
#include <stack>
#include <string>
#include <vector>

#define INF 1e9
```

```
#define ll long long
#define ui unsigned int

const int MAX_NODES = 2100000;

using namespace std;

ui n;
int m;

struct Node {
  ui max, pref, suff;
  int l_c, r_c;
  signed char busy;
} tree[MAX_NODES];

int nodes_cnt = 1;

int build(ui len) {
  if (nodes_cnt + 1 >= MAX_NODES){
    return 0;
  }
  int v = ++nodes_cnt;
  tree[v].max = tree[v].pref = tree[v].suff = len;
  tree[v].l_c = tree[v].r_c = 0;
  tree[v].busy = -1;
  return v;
}

void apply(int v, ui len, int type) {
  tree[v].busy = (signed char)type;
  tree[v].max = tree[v].pref = tree[v].suff = (type == 1 ? len : 0);
}

void push(int v, ui tl, ui tr) {
  if (tl == tr) {
    return;
  }

  ui mid = tl + (tr - tl) / 2;
  if (!tree[v].l_c) {
    tree[v].l_c = build(mid - tl + 1);
  }
  if (!tree[v].r_c) {
    tree[v].r_c = build(tr - mid);
  }

  if (tree[v].busy != -1) {
    apply(tree[v].l_c, mid - tl + 1, tree[v].busy);
    apply(tree[v].r_c, tr - mid, tree[v].busy);
    tree[v].busy = -1;
  }
}

void recalculate(int v, ui tl, ui tr) {
  ui mid = tl + (tr - tl) / 2;
  int l = tree[v].l_c, r = tree[v].r_c;
```

```
  tree[v].pref = tree[l].pref;
  if (tree[l].pref == (mid - tl + 1)) {
    tree[v].pref += tree[r].pref;
  }

  tree[v].suff = tree[r].suff;
  if (tree[r].suff == (tr - mid)) {
    tree[v].suff += tree[l].suff;
  }

  tree[v].max = max({tree[l].max, tree[r].max, tree[l].suff + tree[r].pref});
}

void update(int v, ui tl, ui tr, ui l, ui r, int type) {
  if (l <= tl && tr <= r) {
    apply(v, tr - tl + 1, type);
    return;
  }

  push(v, tl, tr);

  ui mid = tl + (tr - tl) / 2;
  if (l <= mid) {
    update(tree[v].l_c, tl, mid, l, r, type);
  }
  if (r > mid) {
    update(tree[v].r_c, mid + 1, tr, l, r, type);
  }
  recalculate(v, tl, tr);
}

int get_start(int v, ui tl, ui tr, ui k) {
  if (tree[v].max < k) {
    return -1;
  }
  if (tree[v].max == (tr - tl + 1)) {
    return tl;
  }

  push(v, tl, tr);
  ui mid = tl + (tr - tl) / 2;

  if (k <= tree[tree[v].l_c].max) {
    return get_start(tree[v].l_c, tl, mid, k);
  }

  if (k <= tree[tree[v].l_c].suff + tree[tree[v].r_c].pref) {
    return mid - tree[tree[v].l_c].suff + 1;
  }

  return get_start(tree[v].r_c, mid + 1, tr, k);
}

struct Delete {
  ui start, finish;
  bool success = false;
```

```
};

int main() {
  cin >> n >> m;
  int cur;

  tree[1].max = tree[1].pref = tree[1].suff = n;
  tree[1].busy = -1;

  vector<Delete> history(m + 1);

  for (int i = 1; i <= m; i++) {
    cin >> cur;
    if (cur > 0) {
      ui k = (ui)cur;
      int res = get_start(1, 1, n, k);
      cout << res << '\n';
      if (res != -1) {
        update(1, 1, n, (ui)res, (ui)res + k - 1, 0);
        history[i] = {(ui)res, (ui)res + k - 1, true};
      }
    } else {
      int idx = -cur;
      if (history[idx].success) {
        update(1, 1, n, history[idx].start, history[idx].finish, 1);
        history[idx].success = false;
      }
    }
  }
}
```

L. Минимум на отрезке

Для массива строится дерево отрезков, после чего для каждого окна длины k вычисляется минимум на отрезке [i, i + k - 1].

Идея реализации:
- построение дерева отрезков по исходному массиву;
- запрос минимума на отрезке выполняется рекурсивным обходом по пересечению сегментов;
- ответы печатаются для всех окон скользящего диапазона.

Асимптотика:
- время: $O(n + (n - k + 1)\log n)$;
- память: $O(n)$.

Почему так:
- построение дерева занимает $O(n)$: обозначим число листьев через L, а внутренних вершин через I;
- в дереве отрезков каждая внутренняя вершина имеет двух детей, значит число рёбер равно $2I$;
- с другой стороны, для любого дерева число рёбер равно $L + I - 1$;
- приравнивая, получаем $2I = L + I - 1$, откуда $I = L - 1$;
- так как $L = n$, имеем $I = n - 1$, а общее число вершин $V = L + I = 2n - 1$;
- в каждой вершине при построении выполняется константная работа: либо присваивание (лист), либо одна операция min от двух детей (внутренняя вершина);

- следовательно, суммарная трудоёмкость построения равна $O(V) = O(2n-1) = O(n)$;
- один запрос минимума на отрезке стоит $O(\log n)$;
- запросов ровно $n-k+1$, поэтому добавляется $O((n-k+1)\log n)$;
- суммарно получаем $O(n + (n-k+1)\log n)$;
- количество вершин линейно по n, следовательно, объём памяти также имеет порядок $O(n)$.

Почему это оптимально:

- чтобы вывести минимум для каждого окна, нужно получить $n-k+1$ ответов, то есть ниже линейного по числу окон времени опуститься нельзя;
- дерево отрезков даёт универсальный и быстрый способ отвечать на запрос минимума за $O(\log n)$ после линейного построения;
- при фиксированном подходе с независимыми range-min запросами это стандартная асимптотически эффективная структура без лишнего расхода памяти.

```
#include <algorithm>
#include <deque>
#include <iostream>
#include <map>
#include <queue>
#include <set>
#include <stack>
#include <string>
#include <unordered_map>
#include <vector>

#define INF 1000000000
#define ll long long

using namespace std;

struct Segment_tree {
  struct Node {
    ll value;
    ll l, r;

    Node* parent = nullptr;
    Node* left_child = nullptr;
    Node* right_child = nullptr;

    Node() {
      this->value = 0;
      this->l = -1;
      this->r = INF;
    }

    Node(ll l, ll r) : Node() {
      this->l = l;
      this->r = r;
    }
  };

  ll size;
  Node* root;

  Segment_tree(vector<ll>& a) {
    this->size = a.size();
```

```
    this->root = build(a, 0, size - 1, nullptr);
  }

  Node* build(vector<ll>& a, ll l, ll r, Node* parent) {
    Node* node = new Node(l, r);
    node->parent = parent;

    if (l == r) {
      node->value = a[l];
    } else {
      ll mid = (l + r) / 2;
      node->left_child = build(a, l, mid, node);
      node->right_child = build(a, mid + 1, r, node);
      node->value = min(node->left_child->value, node->right_child->value);
    }
    return node;
  }

  ll print(ll l, ll r) { return print(root, l, r); }

  ll print(Node* node, ll l, ll r) {
    if (node == nullptr || r < node->l || node->r < l) {
      return INF;
    }

    if (l <= node->l && node->r <= r) {
      return node->value;
    }

    ll min_left = print(node->left_child, l, r);
    ll min_right = print(node->right_child, l, r);

    return min(min_left, min_right);
  }
};

int main() {
  ll n, k;
  cin >> n >> k;

  vector<ll> mas(n, 0);
  for (int i = 0; i < n; i++) {
    cin >> mas[i];
  }

  Segment_tree tree(mas);

  for (ll i = 0; i < n - k + 1; i++) {
    cout << tree.print(i, i + k - 1) << ' ';
  }
}
```